ALGORITMOS E ESTRUTURA DE DADOS II

Trabalho Prático: Árvore Binária de Busca, Busca Binária, Selection Sort e Análise Teórica de Recorrências

Avaliação

Pode ser realizado individualmente ou dupla.
Todos devem apresentar (Ausência na apresentação sem justificativa formal incorrerá em desconto na nota da dupla) .

Critérios de Avaliação (Nota conjunta das partes de Estruturas e Análises – peso 8 para análise)

1. Corretude do código e funcionalidade (Peso 4 – nota da dupla)
2. Corretude do desenvolvimento da Análise das Recorrências (Peso 3 – nota da dupla)
3. Apresentação com slides (Peso 3 – nota individual)

O que enviar no AVA?

- Todos os códigos
- Relatório com análise em pdf (incluindo as funções analisadas)
- Slides de apresentação em pdf

Submissão até dia 26/10 às 13h
Inicio das apresentações em horário de aula no dia 26/10

1. Visão Geral e Objetivos

O objetivo deste trabalho prático é consolidar os conceitos de estruturas de dados hierárquicas dinâmicas e de paradigmas de projeto de algoritmos através da implementação manual de uma **Árvore Binária de Busca (BST)** em Python, aliada à aplicação do método de **Divisão e Conquista (Busca Binária)** e do algoritmo de ordenação **Selection Sort Recursivo** sobre o percurso dos dados.

Adicionalmente, o trabalho exige a modelagem matemática formal do tempo de execução dos algoritmos recursivos desenvolvidos através de **Equações de Recorrência**, utilizando o Método da Substituição, ou Expansão Algébrica, ou Teorema Mestre.

2. Critérios de Avaliação

A nota total do trabalho é 10,0, distribuída da seguinte forma:

Corretude do Código e Funcionalidade (Peso 4,0): Ausência de nós órfãos ou quebra de ponteiros, implementação estritamente recursiva dos métodos solicitados, tratamento de casos de borda e fidelidade à especificação.

Corretude da Análise de Recorrências (Peso 3,0): Desenvolvimento matemático completo, legível e passo a passo das substituições, somatórios e provas assintóticas.

Apresentação e Defesa (Peso 3,0): Domínio do código e da teoria durante a exposição oral e resposta aos questionamentos.

3. Entregáveis e Prazos

- **Código Fonte:** Arquivo em Python contendo as classes No, ArvoreBinariaBusca, os métodos de busca binária e Selection Sort recursivos, e o menu interativo.

- **Relatório Técnico (PDF):** Documentação completa contendo o código e o desenvolvimento matemático das equações de recorrência.

- **Slides de Apresentação (PDF):** Material visual estruturado para a defesa do trabalho.

- **Data e Hora Limite de Submissão:** 28/10 às 13:00h no AVA.

- **Início das Apresentações:** 28/10 durante o horário de aula.

4. Restrições e Regras de Implementação

- **Orientação a Objetos:** O programa deve ser implementado do zero em Python 3 utilizando o paradigma de Orientação a Objetos (OO).

- **Uso Obrigatório de Recursão:** Funções especificadas como recursivas devem utilizar recursão pura. O uso de laços de repetição (while ou for) em funções marcadas como recursivas invalidará a pontuação da função.

- **Proibição de Listas Nativas Internas:** Não é permitido o uso de estruturas nativas do Python (como listas [] ou dicionários) para armazenar, ordenar ou manipular os dados na árvore. A única exceção é o retorno temporário do vetor

gerado pelo percurso *Em-Ordem* para alimentação da Busca Binária e do Selection Sort.

5. Especificação de Classes e Métodos

5.1. Classe No

Representa cada nó individual da Árvore Binária de Busca. Deve conter os seguintes atributos:

1. chave: Valor inteiro armazenado no nó.
2. esquerda: Ponteiro/referência para o filho à esquerda.
3. direita: Ponteiro/referência para o filho à direita.

5.2. Classe ArvoreBinariaBusca

Gerencia a raiz da árvore e implementa os métodos principais:

Método	Tipo	Descrição
inserir(self, chave)	Público / Auxiliar	Dispara a inserção recursiva a partir da raiz da árvore.
_inserir_recursivo(self, no, chave)	Recursivo (Privado)	Percorre a árvore recursivamente até a posição correta de inserção mantendo a propriedade de BST.
buscar(self, chave)	Público / Auxiliar	Dispara a busca recursiva de uma chave na BST.
_buscar_recursivo(self, no, chave)	Recursivo (Privado)	Navega pela subárvore esquerda ou direita comparando a chave solicitada.

Método	Tipo	Descrição
em_ordem(self, no, resultado=None)	Recursivo	Realiza o percurso Em-Ordem (Esquerda → Raiz → Direita), retornando um arranjo com os elementos.
remover(self, chave)	Recursivo	Remove um elemento tratando os 3 casos: nó folha, nó com 1 filho e nó com 2 filhos (substituição pelo sucessor/antecessor).

5.3. Funções de Ordenação e Busca em Vetor

1. busca_binaria_divisao_conquista(arr, inicio, fim, alvo) (Recursivo): Algoritmo de Divisão e Conquista executado sobre o arranjo ordenado. Divide a área de busca ao meio e realiza chamadas recursivas na metade correspondente.

2. encontrar_indice_minimo_recursivo(arr, i, j) (Recursivo Auxiliar): Localiza recursivamente o índice do menor elemento no sub-arranjo de i até o fim da lista.

3. selection_sort_recursivo(arr, i=0, n=None) (Recursivo): Funciona como a estrutura principal do Selection Sort, selecionando o menor elemento da sublista não ordenada via encontrar_indice_minimo_recursivo e trocando com a posição i.

6. Análise Teórica e Resolução de Recorrências

Exercício 1: Recorrência da Busca na Árvore Binária de Busca

- **Pior Caso (Árvore Degenerada):** Modelar a equação de recorrência T(n) quando a árvore tem altura h = n (equivalente a uma lista encadeada). Resolver por substituição sucessiva demonstrando que T(n) = O(n).T(n) = T(n - 1) + c, com T(1) = d

- **Melhor Caso / Caso Balanceado:** Modelar a equação de recorrência T(n) para uma árvore perfeitamente balanceada (altura $h = \log_2 n$). Resolver aplicando o **Teorema Mestre** para provar que $T(n) = \Theta(\log n).T(n) = T(n/2) + \Theta(1)$

Exercício 2: Recorrência da Busca Binária (Divisão e Conquista)

1. Apresentar a equação de recorrência formal para o pior caso da Busca Binária recursiva:
   $T(n) = T(n/2) + c$, com $T(1) = c_0$

2. Resolver a equação passo a passo pelo Método da Substituição/Expansão até o caso base, demonstrando formalmente a obtenção de $T(n) = O(\log n)$.

Exercício 3: Recorrência do Selection Sort Recursivo

1. Modelar a equação de recorrência T(n) para o Selection Sort recursivo, onde encontrar o menor elemento consome $T_busca(n) = O(n)$ e a chamada recursiva processa (n-1) elementos:
   $$T(n) = T(n - 1) + c \cdot n, \text{ com } T(1) = c_0$$

2. Resolver por expansão algébrica demonstrando formalmente a obtenção da complexidade $T(n) = O(n^2)$.

7. Interface do Usuário (Menu Interativo)

O script principal deve apresentar o seguinte menu em laço while:

```
=== MENU ÁRVORE BINÁRIA DE BUSCA, SELECTION SORT E DIVISÃO E CONQUISTA ===
1. Inserir elemento na BST (Recursivo)
2. Buscar elemento na BST (Recursivo)
3. Remover elemento da BST (Recursivo)
4. Exibir percurso Em-Ordem (Vetor)
5. Ordenar vetor com Selection Sort (Recursivo)
6. Executar Busca Binária por Divisão e Conquista (Vetor)
0. Sair
Digite sua opção:
```